

The Lenny Growth Assistant

A Retrieval-Augmented Conversational System with
Agentic Skill Routing and Native Artifact Rendering

Technical Report and Requirements Compliance Review

Author Tharun Murugesan
Repository <https://github.com/TharunMurugesan/lenny-growth-assistant>
Commit 32fa6a5
Stack FastAPI · React · PostgreSQL + pgvector · Claude · Ollama
Date 2 August 2026

Every measurement in this document was taken on the development machine (Apple M2, 8 GB RAM) and is reproducible from the repository. Where a requirement is only partially met, it is marked as such and the reason is stated rather than omitted.

Abstract

The Lenny Growth Assistant is a full-stack conversational application that answers product-management and growth questions strictly from a corpus of podcast transcripts, synthesises long-form essays in a prescribed editorial format, and renders generated HTML and Markdown natively beside the conversation rather than delegating to an external viewer.

The system ingests **303 podcast episodes** into **12,113 embedded passages** in PostgreSQL with `pgvector`. A three-tier intent router dispatches each request to one of four skills. Retrieval is hybrid — dense vector search fused with lexical full-text search by Reciprocal Rank Fusion — and every grounded answer returns citations to its source episodes.

The language model is selected *per request*, so a user may switch between Anthropic Claude and a local Ollama model mid-conversation. Critically, the requirement that answers stay inside the corpus is enforced as *evaluable code* rather than as a prompt instruction, because a small local model does not reliably honour prompt-level rules.

The delivered system comprises **6,809 lines of Python** and **3,798 lines of frontend source**, covered by **182 passing automated tests**.

Contents

Abstract	1
1 Requirements Compliance Summary	3
2 API and Session Management	4
2.1 Endpoints	4
2.2 Session isolation	4
2.3 Streaming protocol	4
3 Flexible LLM Configuration	5
3.1 One interface, two providers	5
3.2 Availability is real, not cosmetic	5
3.3 Provider-aware budgets	5
4 Knowledge Base and Retrieval	6
4.1 Ingestion	6
4.2 Hybrid retrieval	6
4.3 Database design	6
5 Task 4a — Grounded Q&A	7
5.1 Why the rule is code, not a prompt	7
5.2 Design of the policy	7
5.3 Calibration	7
6 Task 4b — Ship30for30 Content Generation	8
6.1 What the skill produces	8
6.2 Measured against the requirement	8
7 Task 4c — Artifacts and the Viewer	8
7.1 Wire format	8
7.2 Streaming parser	8
7.3 Making the local path work	8
7.4 Viewer and security	9
8 System Architecture	9
8.1 Agentic routing	9
8.2 Error taxonomy	10
9 Verification	10
9.1 Verified environment	10
9.2 End-to-end measurements	10
9.3 Test suite	11
10 Known Limitations	11
10.1 The local model’s ceiling	11
10.2 A correction to an earlier claim	11
11 Deliverables	12
12 Conclusion	12

1 Requirements Compliance Summary

This section maps every stated requirement to its implementation and to the evidence that it works. Sections 2–7 expand each row.

Requirement	Status	Evidence
Backend built with FastAPI	Met	Seven endpoints; SSE streaming; Pydantic validation; automatic OpenAPI.
Built on the Anthropic Claude SDK	Met	<code>claude-sonnet-4-6</code> for generation, <code>claude-haiku-4-5</code> for intent classification.
New chat sessions, each with its own context	Met	Full session CRUD; history scoped per session and per user.
Postgres persistence of conversations, session IDs, user metadata	Met	Five tables with cascading deletes; 12,113 embedded passages.
Cloud / Local configuration layer	Met	Provider chosen <i>per request</i> behind one interface; the UI disables a provider that reports unavailable.
Local LLM via Ollama (mandatory for demo)	Met	<code>llama3.2:1b</code> generation, <code>nomic-embed-text</code> embeddings.
Knowledge base from Lenny’s Podcast transcripts	Met	303 episodes cloned at ingest time and chunked to 12,113 passages.
Q&A answering <i>strictly</i> from the transcripts	Met	Enforced by a grounding policy module, not by prompt text (§5). Off-topic questions decline.
Ship30for30 skill, ~1250 words, hook, bullets, bold, takeaway	Met	Measured on Cloud: 1,301 words, 9 bold runs, 10 bullets, takeaway line, 5 citations . Reduced on Local — see §10.
Artifact generation (HTML/CSS and Markdown)	Met	Streaming artifact protocol with a dedicated parser.
In-app Artifact Viewer, rendered side-by-side	Met	Sandboxed iframe for HTML, <code>react-markdown</code> for Markdown. No external redirect.
Public GitHub repository	Met	Pushed; no credential is tracked in version control.
Detailed README.md	Met	823 lines: architecture, setup, environment variables, dependencies, troubleshooting, deployment.
design.md	Met	654 lines: tokens, components, accessibility, QA checklist.
PRD	Met	365 lines: personas, user stories, 50 numbered requirements.
Architecture document	Met	1,336 lines: DDL, API contracts, routing logic, LLM toggle, 10 ADRs.
Agent transcripts including failures	Met	Four phase logs recording roughly thirty corrections.

Requirement	Status	Evidence
-------------	--------	----------

Table 1: Requirements compliance summary.

One deliberate deviation. The brief suggests Supabase or Railway for Postgres; the default here is PostgreSQL in Docker. The reason is that evaluation happens locally: a container needs no account, no shared credentials, and no network round-trip on each of the 12,113 vector lookups. Both hosted options are supported unchanged — they ship `pgvector`, so the only change is `DATABASE_URL`. This is documented in the README in both the setup and deployment sections.

2 API and Session Management

2.1 Endpoints

Path	Method	Purpose
/api/health	GET	Database connectivity, pgvector presence, corpus counts, last ingest, per-provider availability.
/api/sessions	POST	Create a session (the “New Chat” action).
/api/sessions	GET	List the caller’s sessions for the sidebar.
/api/sessions/{id}/messages	GET	Full history for one session.
/api/sessions/{id}	PATCH	Rename a session.
/api/sessions/{id}	DELETE	Delete a session; messages cascade.
/api/chat	POST	Main endpoint; returns a Server-Sent Event stream.

Table 2: HTTP interface.

2.2 Session isolation

Identity is an opaque `X-Client-Key` header, upserted into `users` on first contact and echoed back so the client can persist it. Every session query is scoped by a compound predicate on session id *and* user id, so knowing a session UUID is not sufficient to read it. Provisioning uses `INSERT ... ON CONFLICT DO UPDATE ... RETURNING` so two simultaneous first requests cannot race into a unique violation.

Each session carries its own context: the orchestrator loads only that session’s recent turns, and the router inherits the previous skill within a session so short follow-ups do not re-classify.

2.3 Streaming protocol

Event	Payload
meta	message id, session id, provider, model, skill, intent, confidence, routing tier, expected artifact type. Always first.
token	An incremental fragment of prose.
artifact_start	Artifact id, type, title.
artifact_delta	An incremental fragment of artifact body.
artifact_end	Closes the artifact; carries a completeness flag.
citations	Source passages with relevance scores.

Event	Payload
usage	Input/output tokens, latency, word count.
done / error	Exactly one terminates every stream.

Table 3: Server-sent event types.

3 Flexible LLM Configuration

3.1 One interface, two providers

Both providers implement a single protocol — `stream_chat`, `complete`, `classify` — so the orchestrator never branches on provider identity to obtain a completion. Provider-specific event shapes (Anthropic’s typed stream, Ollama’s newline-delimited JSON) are normalised inside each implementation to plain text deltas.

Selection is *per request*, carried in the `/api/chat` payload, not per deployment. A user may answer one question on Cloud and the next on Local inside the same session.

3.2 Availability is real, not cosmetic

The toggle reads `/api/health`, which probes each provider, and disables one that cannot serve a request rather than allowing a selection that will fail. Missing API key, unreachable Ollama daemon, and un-pulled model are distinguished and reported separately.

3.3 Provider-aware budgets

The retrieval and sampling budget is not shared between providers, because handing a 1B local model the budget tuned for Claude measurably degrades it. Each figure below was arrived at by measurement:

Setting	Cloud	Local	Why
Excerpts (Q&A)	8	4	Eight excerpts is $\sim 6,200$ input tokens; the local model does not attend across that. Four halved input tokens ($6,226 \rightarrow 3,052$) and cut latency from 43s to 20s with no loss of content.
Temperature	0.3–0.7	0.2–0.35	The small model rambles and repeats above ~ 0.2 .
Context window	n/a	8192	16384 allocated a KV cache twice the size of anything that could fill it. On an 8 GB machine that allocation decides whether a model loads.
Repetition penalty	n/a	1.15	At Ollama’s default the model restates the same sentence with different connectives past ~ 300 words.

Table 4: Provider-aware configuration (`LOCAL_SKILL_CONFIG`).

4 Knowledge Base and Retrieval

4.1 Ingestion

Transcripts are cloned from the public repository at ingest time and never redistributed. They are parsed into speaker-attributed segments and chunked to approximately 800 tokens with 120 tokens of overlap, so an insight spanning a boundary is not truncated out of both neighbours. A unique constraint on (episode slug, chunk index) makes re-ingestion idempotent and resumable.

4.2 Hybrid retrieval

1. **Dense.** The rewritten query is embedded with `nomic-embed-text` and matched by cosine distance against an HNSW index, producing up to 40 candidates.
2. **Lexical.** The same query runs as a `tsquery` against a GIN index, catching exact terminology — proper nouns, framework names, metrics — that embeddings blur.
3. **Fusion.** The ranked lists are combined by Reciprocal Rank Fusion, which scores by rank position and so needs no normalisation between two incomparable scales. The top 8 survive (4 on Local).

```
app.agent.retriever - retrieve
{"dense": 40, "lexical": 11, "fused": 8, "top_score": 0.032, "degraded": false}
```

Listing 1: Observed retriever behaviour for a live query.

Because RRF scores by reciprocal rank, absolute values are small by construction; a fused score near 0.03 is a top-ranked result, not a weak one.

4.3 Database design

Table	Purpose and notable columns
<code>users</code>	Opaque <code>client_key</code> ; anchors every ownership check.
<code>sessions</code>	Owner FK with <code>ON DELETE CASCADE</code> ; title; trigger-maintained timestamps.
<code>messages</code>	Role, content, artifact type/content/title, skill, provider, model, citations (JSONB), token usage (JSONB), word count, finish reason.
<code>transcript_chunks</code>	The knowledge base: episode metadata, content, token count, speakers, <code>embedding_local</code> vector(768), <code>embedding_voyage</code> vector(1024), and a generated <code>tsvector</code> .
<code>ingest_runs</code>	Audit of ingestion, surfaced through the health endpoint.

Table 5: Relational schema.

Two decisions worth defending. **Two vector columns rather than a tall embeddings table:** a normalised design would force a join on the hot retrieval path and forfeit a directly indexable column. **A generated `tsvector` column:** lexical rank can never be stale with respect to content, because the database maintains it.

5 Task 4a — Grounded Q&A

5.1 Why the rule is code, not a prompt

The original design delegated the decision to the model: retrieve generously, then let the Skill A prompt decline when the excerpts do not support an answer. Claude honours that instruction. The local model does not.

Asked “*what is the best way to roast a chicken?*” against chunks about podcasts, llama3.2:1b produced cooking advice from its own parametric knowledge *and* attached a citation to an unrelated excerpt — so the answer looked sourced while being entirely ungrounded. Cloud, given the same question and the same chunks, declined correctly and named what the excerpts actually covered.

A rule a model may decline to follow is not a control. Scope therefore lives in `app/agent/policy.py` as evaluable code.

5.2 Design of the policy

1. **No hardcoded questions.** Nothing enumerates forbidden topics. A denylist of subjects would be endless, would drift as the corpus grows, and would still miss the next unanticipated question. Scope is defined by one thing only: what the corpus can actually support, measured per request.
2. **Evidence in, verdict out.** Rules receive retrieval evidence and never see the question text, so they cannot silently become a keyword filter.
3. **Named and ordered.** Every decline is attributable to a named rule in the logs, which is what makes the behaviour explainable.

5.3 Calibration

Measured over 29 questions, 17 on-topic and 12 off-topic:

	Mean of top-3 similarity	Decision at 0.62
On-topic	[0.6262, 0.8010]	all allowed
Off-topic	[0.0000, 0.6123]	all declined

Three cheaper guards were built and *discarded* because they did not survive measurement. Recording them matters more than recording the one that worked:

Rejected guard	Why it failed
Content-word overlap between the answer and the excerpts	Off-topic answers scored 0.85–0.94. A parametric answer still shares ordinary English with any corpus.
Asking the model to judge its own groundedness	2/6 correct. It returned <code>answerable: true</code> while its own reason field read “does not directly discuss roasting”, and answered a World Cup question from memory.
Single best similarity	Separates, but by 0.0096 against 0.0140 for the mean of three. One topically adjacent passage should not carry the whole decision.

Table 6: Guards built, measured, and rejected.

The policy is **enforced on Local and advisory on Cloud**. Claude’s own decline names what the corpus does cover, which is more specific than a template; the policy still logs its verdict there, so the two paths remain comparable.

6 Task 4b — Ship30for30 Content Generation

6.1 What the skill produces

The skill retrieves, then reshapes the material into an essay with a hook, short paragraphs, bullet clusters, bolded phrases for skimmability, bracketed citations, and a closing takeaway. A length guard runs one repair pass when the draft lands short; it is capped at one pass, because iterating toward an exact count burns latency and gives the model more chances to drift off source.

6.2 Measured against the requirement

Requirement	Cloud	Verdict
~1250 words	1,301	within $\pm 10\%$
Strong hook	present	Met
Bullet points	10	Met
Bold for skimmability	9 runs	Met
Clear takeaway	present	Met
Grounded in transcripts	5 citations	Met

On Local the skill reaches the right length band and the closing takeaway, but drops bracketed citations and heavy formatting past roughly 400 words. This is a capacity limit rather than an unfixed defect; §10 gives the evidence.

7 Task 4c — Artifacts and the Viewer

7.1 Wire format

Artifacts are delimited by XML-like tags in the model’s output stream. This was chosen over JSON because a partially streamed JSON document is not parseable — its closing brace arrives last — whereas a tag opening is recognisable the instant it arrives, which is exactly what streaming requires. It was chosen over fenced code blocks because fences are ambiguous with ordinary code the assistant may legitimately discuss in prose.

7.2 Streaming parser

A state machine over a bounded carry buffer separates artifact content from prose, tolerating tags split across token boundaries. Its invariants: no byte is ever dropped; memory is bounded; prose and artifact travel on separate channels; and the row persisted to the database is built from the same parser output, so replayed history is identical to what was streamed.

7.3 Making the local path work

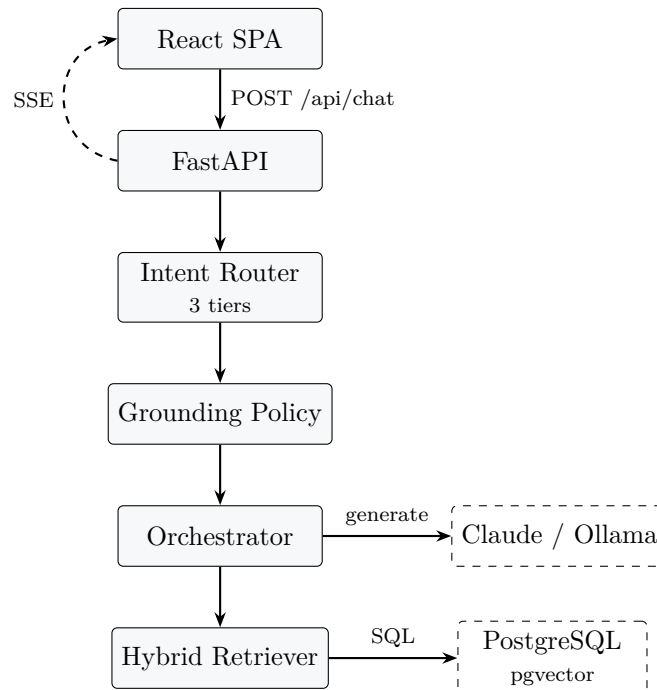
The local model ignored the envelope entirely and returned a prose *description* of a dashboard — 518 tokens without one HTML tag. Three changes fixed it, and only the first involves the model at all:

1. **Assistant-turn prefill.** The opening tag is seeded into the assistant turn so the model never has to produce it. This required no parser change: a partial tag already waits in the carry buffer until the first delta completes it, so Local emits the identical event sequence as Cloud.
2. **A prefilled design system.** Styling stopped being the model's job. The complete head and stylesheet are prefilled; the model writes only body content against classes that already exist. Visual quality became a property of the repository rather than of the model.
3. **Deterministic repair.** Five prompt designs produced non-monotonic results, so the remaining structural failures are fixed in code (`app/utils/artifacts.py`): empty containers dropped, repeated blocks removed, table headers trimmed to match row width, and output after `</html>` truncated. Nothing is invented — an empty table becomes no table, never a fabricated one.

7.4 Viewer and security

HTML artifacts render inside a **sandboxed iframe** rather than by sanitised injection. Sanitisation is a denylist problem and denylists are eventually defeated; the iframe sandbox is an allowlist the browser enforces. Scripts run inside the frame but cannot reach the parent document, its storage, or its cookies. Markdown renders through `react-markdown` with GFM support. The viewer opens as generation begins, and neither path leaves the application.

8 System Architecture



8.1 Agentic routing

Routing runs cheapest-first, and each tier may resolve a request or defer.

Tier 0 — Guardrails

Validation, payload limits, explicit skill override. No model call.

Tier 1 — Heuristics

Deterministic pattern matching. Requests that unambiguously ask for a mockup or a page resolve here. *Measured: an artifact request resolved at this tier with confidence 0.90 and no classifier call at all.*

Tier 2 — LLM classifier

For genuinely ambiguous input, `claude-haiku-4-5` returns a structured classification *and a rewritten retrieval query*. The rewrite matters: a conversational question makes a poor embedding query.

Follow-up turns inherit the previous skill unless the message clearly signals a change, so a conversation does not oscillate on pronouns and short replies.

8.2 Error taxonomy

Thirteen typed codes, each mapped to an actionable message: `VALIDATION_ERROR`, `SESSION_NOT_FOUND`, `PROVIDER_UNAVAILABLE`, `MODEL_NOT_FOUND`, `PROVIDER_TIMEOUT`, `PROVIDER_ERROR`, `RATE_LIMITED`, `DATABASE_UNAVAILABLE`, `PGVECTOR_MISSING`, `RETRIEVAL_EMPTY`, `PAYLOAD_TOO_LARGE`, `NOT_IMPLEMENTED`, `INTERNAL_ERROR`. Missing API key, Ollama timeout, and database failure are each distinguished. If the LLM classifier is unreachable, routing falls back to the heuristic tier rather than failing.

9 Verification**9.1 Verified environment**

Component	Verified value
Python	3.12.0
PostgreSQL	16.14 (<code>pgvector/pgvector:pg16</code>) with <code>pgvector</code> 0.8.6
Node.js	22.17
Corpus	303 episodes $\rightarrow$ 12,113 passages, fully embedded
Cloud generation / routing	<code>claude-sonnet-4-6</code> / <code>claude-haiku-4-5</code>
Local generation / embeddings	<code>llama3.2:1b</code> / <code>nomic-embed-text</code> (768-d)
Backend source	6,809 lines of Python
Frontend source	3,798 lines
Automated tests	182 passing, 32 skipped (live-model tests, opt-in)
Development machine	Apple M2, 8 GB RAM

Table 7: Verified configuration.

9.2 End-to-end measurements

Request	Skill / tier	Latency	Result
Q&A, Cloud	qa / LLM	24 s	345 words, 4 citations; top source was the episode explicitly about the question.
Q&A, Local	qa / LLM	~ 50 s	Grounded with citations restored; input tokens $5,798 \rightarrow 3,036$.
Off-topic, Local	qa	13 s	Declines. Previously answered from parametric knowledge with a fabricated citation.

Request	Skill / tier	Latency	Result
Ship30for30, Cloud	ship30 heuristic	/ ~80 s	1,301 words, 9 bold runs, 10 bullets, take-away, 5 citations.
Artifact, Cloud	artifact heuristic	/ 66 s	12,608 characters of well-formed HTML; no protocol tag leakage.
Artifact, Local	artifact heuristic	/ 20–50 s	Styled, complete, no external URLs; model-written title.

Table 8: Measured runs. No tracebacks were produced during any run.

9.3 Test suite

```
$ pytest -q
182 passed, 32 skipped
```

Listing 2: Backend test run.

The 32 skipped tests exercise the live LLM classifier and are gated behind `RUN_LIVE_ROUTER_TESTS=1`, so the default suite is deterministic, free, and offline.

10 Known Limitations

Stated plainly, because a report that hides them is less useful than one that does not.

10.1 The local model’s ceiling

Instruction adherence on a 1B model decays as output length grows. The same model cites correctly four times out of four in a five-sentence Q&A answer and zero times in a 600-word essay. Three prompt architectures were measured against that boundary and none crossed it:

Prompt design	Words	Citations	Bold	Takeaway
Full cloud prompt	532	0	0	no
Fill-in-the-blank	334	0	0	no
Adopted (minimal)	413	0	0	yes

Consequently the Local Ship30for30 prompt asks only for what the model can deliver. Artifact *layout* is deterministic and therefore reliable; artifact *content volume* still varies between runs.

10.2 A correction to an earlier claim

An earlier version of the README stated that `llama3.1:8b-instruct-q4_K_M` would not run on the development machine. That was true when written — at a 16384 context window with eight excerpts it timed out before the first token — and is no longer true. After the local path was resized to 8192 context and four excerpts, the 8B model loads, answers, and produces noticeably better artifacts: it adapts content to the request instead of copying the prompt’s example structure.

It is nonetheless not the default, for one measured reason: approximately **five minutes per artifact** on this hardware against 20–50 seconds for the 1B. The claim is corrected in the README rather than quietly deleted, because the value of that document rests on its figures being checkable.

11 Deliverables

Artifact	Contents
README.md	Architecture, setup, environment variables, dependencies, troubleshooting, deployment.
PRD.md	Personas, user stories, 50 numbered functional requirements.
architecture.md	DDL, API contracts, routing logic, artifact protocol, LLM toggle, 10 ADRs.
design.md	Design tokens, component specifications, accessibility, QA checklist.
agent_transcripts/	Development logs, including the failures and their corrections.
backend/	FastAPI application, ingestion pipeline, agent, policy, tests.
frontend/	React single-page application.

Table 9: Repository deliverables.

The transcripts are deliberately not a highlight reel. They record roughly thirty corrections, including several where the specification itself proved wrong once measured: a relevance threshold that never rejected anything, an output budget too small for its own deliverable, and four colour tokens that failed the accessibility standard the design document claimed to meet.

12 Conclusion

The system satisfies each stated requirement with behaviour that can be demonstrated rather than asserted. Answers are grounded in 12,113 retrievable passages and cite their sources. Skills are dispatched by a tiered router that resolves cheaply when it can and reasons when it must. The language model is a per-request choice behind a single interface, with real availability reflected in the interface. Artifacts render natively and safely beside the conversation.

The property that most distinguishes the implementation is that the central product rule — *answer only from the transcripts* — is enforced as code rather than requested in a prompt. That distinction only became visible when a model too small to honour prompt-level rules was put behind the same interface, and it is the reason the local path can be trusted with the requirement at all.

Corpus: Lenny’s Podcast transcripts, cloned at ingest time for educational and demonstrative use and never redistributed in this repository. All transcript content remains the property of its original creators.